ANALISIS DEL EJERCICIO 1

INSTRUCCIONES:

- Indicación: Ejecuta estimate_pi con 1, 2, 4 y 8 procesos.
- Visualización: Grafica la aceleración (speedup) vs. número de procesos
- Análisis: ¿Por qué la aceleración no es perfectamente lineal? (Considera sobrecostos de comunicación, sincronización o la ley de Amdahl).

Luego de ejecutar el código un par de veces y revisar los resultados de la aceleración paralela para la estimación de π, lo primero que se nota a simple vista es que la línea del *speedup* no crece de forma lineal como se esperaría en teoría, sino que empieza a disminuir el crecimiento, especialmente después de 4 procesos.

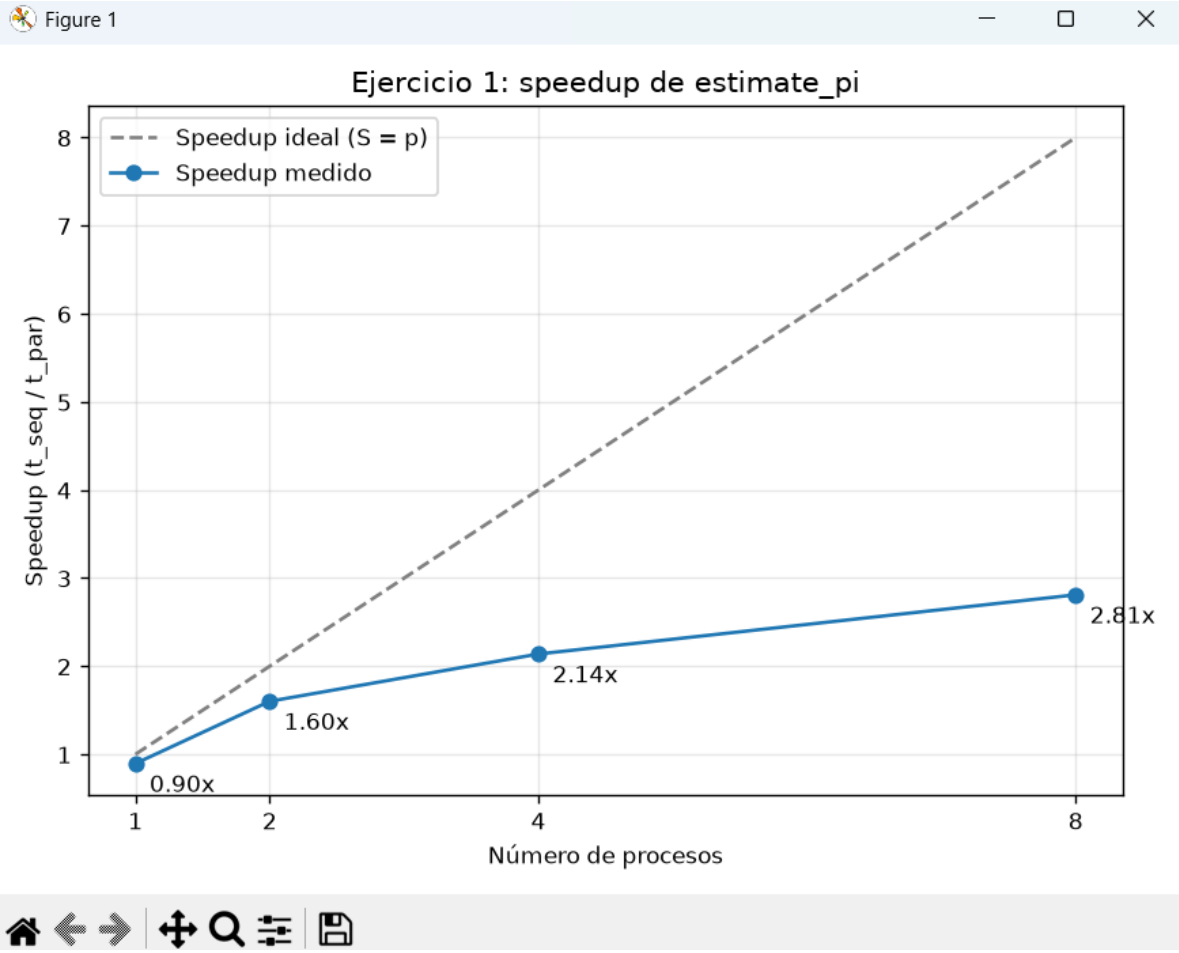
Luego del análisis a fondo de los datos, investigando por qué pasa esto encontré que esto pasa por una combinación de factores propios de cómo funciona la computación paralela en la práctica:

Empezando con que chocar contra la ley de Amdahl es prácticamente inevitable, esta ley describe el límite teórico de aceleración de un sistema cuando solo una parte del mismo puede mejorarse o paralelizarse, en este caso aunque calcular los puntos de Monte Carlo es un trabajo totalmente independiente, el programa entero tiene una parte que no se puede repartir como crear el Pool, arrancar los procesos, enviar las instrucciones y recolectar los resultados finales.

Un detalle que me llama la atención es lo que pasa cuando usamos un solo proceso, al correr la prueba con $p = 1$, el programa fue un poco más lento que hacerlo de forma secuencial, luego de revisar y consultar he llegado a la conclusión de que esto ocurre por el sobrecosto de creación conocido como “overhead”, ya que Inicializar el Pool toma una fracción de tiempo que no aporta nada útil si solo usas un proceso.

Durante la investigación me di cuenta de que también influye bastante la sincronización y el balance de carga. Al usar pool.map con 8 tareas grandes, el programa principal se queda esperando obligatoriamente a que termine el proceso más lento, en teoría, con 8 procesos el tiempo debería ser mínimo, pero en la práctica medimos más tiempo del esperado debido a los pequeños retrasos y los tiempos muertos de espera entre los hilos.

GRAFICA SPEEDUP VS CANTIDAD DE PROCESOS



Aquí podemos ver grafica de una ejecución donde la línea azul representa el crecimiento del speedup y es posible observar como el crecimiento se reduce cada vez más, la línea punteada representa el speedup ideal teórico el cual crece linealmente por cantidad de procesos